



UNIVERSIDADE ESTADUAL DE MARINGÁ – UEM
CENTRO DE TECNOLOGIA – CTC
DEPARTAMENTO DE INFORMÁTICA – DIN

EDUARDO ANGELO ROZADA MINHOLI – RA 134932
NUNO MIGUEL MENDONÇA ABILIO – RA 132830

TRABALHO - 3ª AVALIAÇÃO PERIÓDICA - ALGORITMOS EM
GRAFOS
“AVALIAÇÃO COMPUTACIONAL DE PRIM VS KRUSKAL”

Maringá-PR
2025

EDUARDO ANGELO ROZADA MINHOLI – RA 134932
NUNO MIGUEL MENDONÇA ABILIO – RA 132830

**TRABALHO - 3ª AVALIAÇÃO PERIÓDICA - ALGORITMOS EM
GRAFOS**
“AVALIAÇÃO COMPUTACIONAL DE PRIM VS KRUSKAL”

*Relatório de avaliação computacional apresentado como
requisito total à aprovação na disciplina de “Algoritmos
em Grafos” do Curso de Bacharelado em Ciência da
Computação da Universidade Estadual de Maringá.*

Prof. Ademir Aparecido Constantino .

Sumário

	Páginas
1 Introdução	4
2 Metodologia e Implementação	4
2.1 Algoritmos Implementados	5
2.1.1 Algoritmo de Kruskal	5
2.1.2 Algoritmo de Prim	5
2.2 Ambiente de Testes e Métricas	6
3 Resultados Experimentais	6
3.1 Instâncias de Teste	6
3.2 Desempenho de Tempo	7
3.2.1 Visualização Gráfica do Tempo de Execução	7
3.3 Consumo de Memória	8
3.3.1 Visualização Gráfica do Consumo de Memória	9
4 Discussão dos Resultados	10
4.1 Comparativo: Variações do Kruskal	10
4.2 Comparativo: Prim Lista vs. Prim Heap	10
4.3 Prim vs. Kruskal	11
5 Conclusão	11

1 Introdução

O problema da Árvore Geradora Mínima é um dos problemas fundamentais na teoria dos grafos e na otimização combinatória. Dado um grafo não direcionado e ponderado, o objetivo consiste em encontrar um subconjunto de arestas que conecte todos os vértices do grafo, sem formar ciclos, e cuja soma dos pesos das arestas seja a mínima possível. Este problema possui diversas aplicações práticas, desde o planejamento de redes de computadores e distribuição elétrica até a clusterização de dados.

O presente trabalho tem como objetivo principal avaliar e comparar o desempenho computacional de duas abordagens clássicas para a solução deste problema: o algoritmo de Prim e o algoritmo de Kruskal. A análise foca em métricas de eficiência, especificamente o tempo de execução e o consumo de memória, bem como a consistência dos resultados obtidos por diferentes implementações. Além disso, busca-se investigar como o desempenho desses algoritmos varia conforme o crescimento do tamanho das instâncias (número de vértices e arestas).

Para este estudo, foram utilizadas instâncias de grafos onde os vértices são identificados por coordenadas no sistema UTM (x_i, y_i) . Consequentemente, o custo (peso) de cada aresta é definido implicitamente pela distância Euclidiana entre os pares de vértices, calculada pela fórmula $d(v_i, v_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.

As implementações foram realizadas na linguagem Python, utilizando apenas estruturas de dados nativas, sem o auxílio de bibliotecas externas específicas para manipulação de grafos, conforme os requisitos da avaliação. Foram desenvolvidas e testadas variações dos algoritmos para isolar o impacto de diferentes estruturas de dados, como o uso de *Heap* versus busca linear no algoritmo de Prim, e diferentes otimizações de estrutura no algoritmo de Kruskal.

Este relatório está organizado da seguinte maneira: a Seção 2 detalha a metodologia empregada, descrevendo as estruturas de dados e as variações dos algoritmos implementados, bem como o ambiente de testes; a Seção 3 apresenta os dados brutos obtidos nos experimentos; a Seção 4 discute e analisa os resultados, comparando a eficiência das abordagens frente ao crescimento das instâncias; e, por fim, a Seção 5 apresenta as conclusões finais e limitações observadas durante o estudo.

2 Metodologia e Implementação

Para atingir os objetivos propostos, foram implementados os algoritmos de Kruskal e Prim utilizando a linguagem de programação Python. Seguindo os requisitos da avaliação, não foram utilizadas bibliotecas externas de grafos, restringindo-se o desenvolvimento ao uso de estruturas de dados nativas da linguagem e bibliotecas padrão. Apenas a biblioteca `psutil` foi empregada externamente para a coleta de métricas de memória.

2.1 Algoritmos Implementados

Foram desenvolvidas múltiplas versões de cada algoritmo para isolar e analisar o impacto de diferentes otimizações e estruturas de dados.

2.1.1 Algoritmo de Kruskal

O algoritmo de Kruskal seleciona arestas em ordem crescente de peso, adicionando-as à Árvore Geradora Mínima (MST) caso não formem ciclos. A detecção de ciclos e a união de componentes conexos foram gerenciadas pela estrutura de dados *Union-Find* (ou Conjunto Disjunto). Foram implementadas três variações:

- **Kruskal Simples:** Utiliza uma implementação ingênua do *Union-Find*, onde a operação *find* não realiza compressão de caminho e a operação *union* anexa arbitrariamente uma árvore à outra, podendo gerar estruturas degeneradas (listas ligadas) com complexidade de busca linear.
- **Kruskal com Rank:** Implementa a otimização de *Union by Rank*, garantindo que a árvore de menor altura seja sempre anexada à raiz da árvore de maior altura, mantendo a estrutura balanceada.
- **Kruskal Otimizado:** Combina *Union by Rank* com a *Path Compression* (Compressão de Caminho). Durante a operação *find*, os nós visitados são reconectados diretamente à raiz, achatando a estrutura e garantindo complexidade de tempo quase constante para as operações subsequentes.

2.1.2 Algoritmo de Prim

O algoritmo de Prim constrói a MST a partir de um vértice inicial arbitrário, expandindo a fronteira de vértices explorados ao selecionar a aresta de menor custo que conecta um vértice da árvore a um vértice fora dela. Duas estratégias foram adotadas para a seleção do vértice de menor custo:

- **Prim Lista (Busca Linear):** Utiliza uma lista/conjunto para controlar os vértices não visitados. A busca pelo vértice de menor custo é realizada iterando linearmente sobre todos os candidatos, resultando em uma complexidade assintótica de $O(N^2)$.
- **Prim Heap (Fila de Prioridade):** Utiliza uma *Min-Heap* binária (implementada via módulo *heapq*) para gerenciar as arestas de fronteira. Isso permite extrair o mínimo em tempo logarítmico, resultando em uma complexidade $O(M \log N)$, mais adequada para grafos esparsos.

2.2 Ambiente de Testes e Métricas

Os experimentos foram conduzidos em um ambiente controlado para garantir a consistência das medições. As métricas de desempenho coletadas foram:

- **Tempo de Execução:** Medido utilizando a função `time.perf_counter()` da biblioteca padrão, que oferece alta precisão para medir intervalos de tempo de processamento (tempo de relógio), capturando o tempo total desde o início da execução do algoritmo até a obtenção da MST completa.
- **Consumo de Memória:** O uso de memória foi monitorado através da biblioteca `psutil`, capturando a métrica RSS (*Resident Set Size*) do processo. Foi calculado o diferencial entre a memória residente antes e após a execução do algoritmo para estimar o impacto específico de cada implementação.

Para validar a corretude das implementações, o custo total da MST gerada por cada algoritmo foi comparado. Em todas as instâncias onde os algoritmos convergiram, os custos obtidos foram idênticos, confirmando a precisão das soluções encontradas.

3 Resultados Experimentais

Nesta seção, apresentamos os dados quantitativos obtidos durante a execução dos algoritmos. Os testes foram realizados em 6 instâncias de grafos com características variadas de tamanho e densidade, utilizando as métricas de tempo de execução e consumo de memória.

3.1 Instâncias de Teste

A Tabela 1 resume as características dos grafos utilizados, detalhando o número de vértices (V), o número de arestas (E) e o custo total da Árvore Geradora Mínima encontrada. Observa-se uma variação significativa de escala, desde grafos menores (Grafo 2, com cerca de 5 mil vértices) até grafos de grande porte (Grafo 5, com mais de 212 mil vértices).

Tabela 1. *Características das Instâncias de Teste e Custo da MST*

Instância	Vértices (V)	Arestas (E)	Custo MST
Grafo 1	8.980	12.629	3015372,43
Grafo 2	5.152	7.086	2641212,29
Grafo 3	37.128	54.867	3061889,58
Grafo 4	12.368	16.965	4794641,15
Grafo 5	212.341	307.292	16391318,64
Grafo 6	154.938	214.893	12342510,35

3.2 Desempenho de Tempo

A Tabela 2 apresenta os tempos de execução (em segundos) para cada algoritmo em todas as instâncias. Nota-se uma disparidade expressiva no algoritmo *Prim Lista* para os grafos maiores, chegando a quase 20 minutos (1170s) no Grafo 5. O *Kruskal Simples* falhou nas duas maiores instâncias (Grafo 5 e 6).

Tabela 2. Tempo de Execução (em segundos)

Grafo	K. Simples	K. Rank	K. Otimizado	Prim Lista	Prim Heap
Grafo 1	0,0785	0,0711	0,0691	1,6773	0,0877
Grafo 2	0,0402	0,0406	0,0410	0,5525	0,0491
Grafo 3	0,4494	0,3192	0,3475	33,4218	0,4137
Grafo 4	0,0961	0,0940	0,0974	2,7068	0,1234
Grafo 5	Falhou	2,0787	2,0924	1.170,3835	2,8712
Grafo 6	Falhou	1,5000	1,4898	616,5304	1,9942

3.2.1 Visualização Gráfica do Tempo de Execução

A seguir, apresentamos os gráficos comparativos de tempo para todas as 6 instâncias. É importante notar a variação da escala no eixo vertical (tempo em segundos) entre os grafos menores e os maiores.

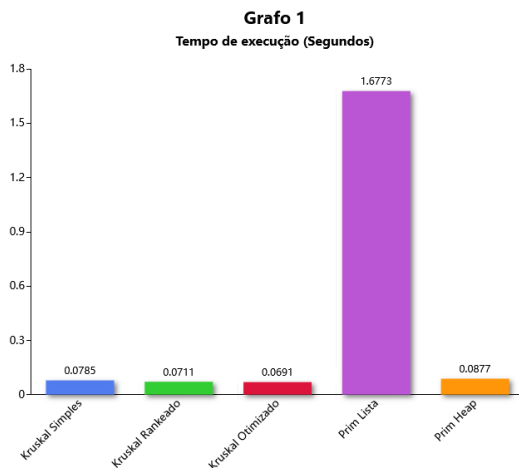


Figura 1. Tempo - Grafo 1

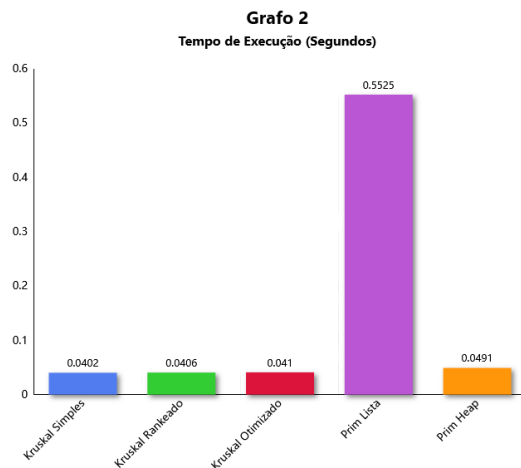


Figura 2. Tempo - Grafo 2

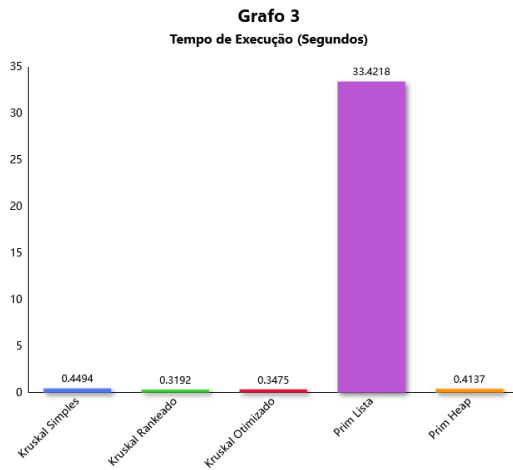


Figura 3. Tempo - Grafo 3

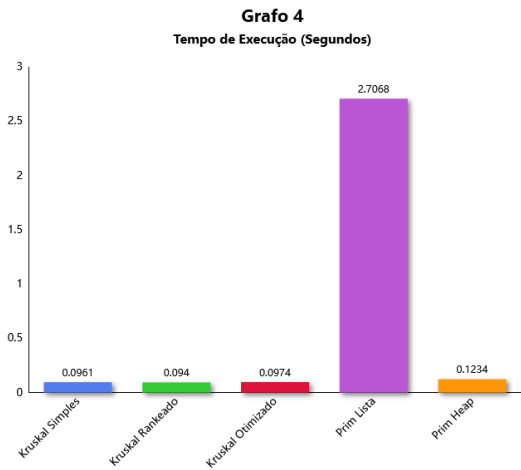


Figura 4. Tempo - Grafo 4

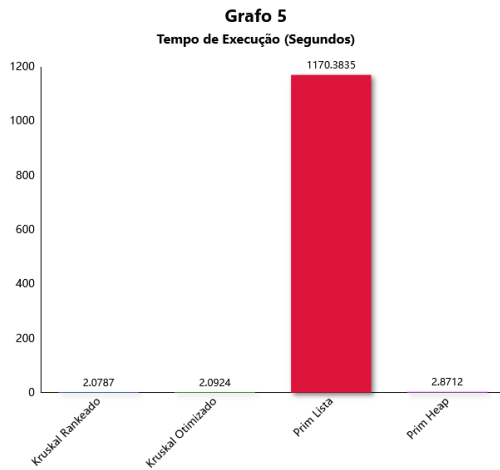


Figura 5. Tempo - Grafo 5

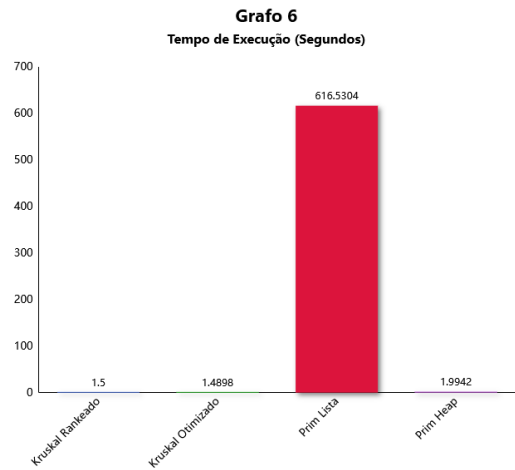


Figura 6. Tempo - Grafo 6

Nota: Nos Grafos 5 e 6, o algoritmo Kruskal Simples não aparece pois falhou, e a barra do Prim Lista é significativamente maior que as demais.

3.3 Consumo de Memória

A Tabela 3 detalha o aumento do uso de memória RAM (em MB). O Prim Heap consumiu consistentemente mais memória nas grandes instâncias (aprox. 193 MB no Grafo 5), seguido pelo Prim Lista. As implementações de Kruskal demonstraram maior eficiência espacial.

Tabela 3. Aumento do Uso de Memória (em MB)

Grafo	K. Simples	K. Rank	K. Otimizado	Prim Lista	Prim Heap
Grafo 1	4,70	4,90	4,98	7,89	8,62
Grafo 2	3,06	2,66	2,67	4,29	4,96
Grafo 3	19,61	19,42	19,40	33,18	35,55
Grafo 4	6,74	6,49	6,46	10,60	11,85
Grafo 5	-	112,30	113,28	184,85	193,33
Grafo 6	-	77,29	77,44	131,34	135,57

3.3.1 Visualização Gráfica do Consumo de Memória

A seguir, apresentamos os gráficos comparativos de uso de memória para todas as 6 instâncias.

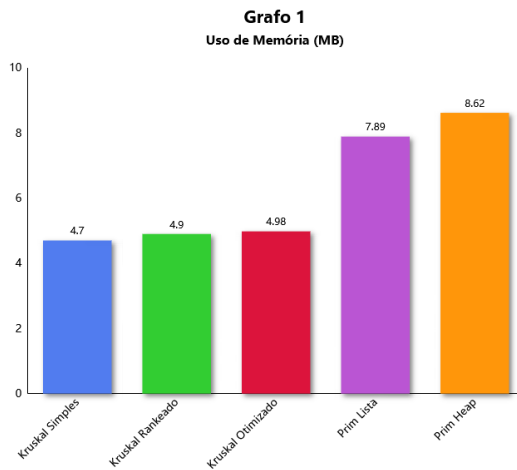


Figura 7. Memória - Grafo 1

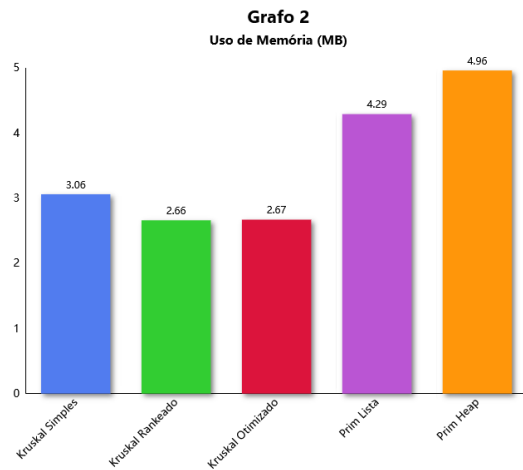


Figura 8. Memória - Grafo 2

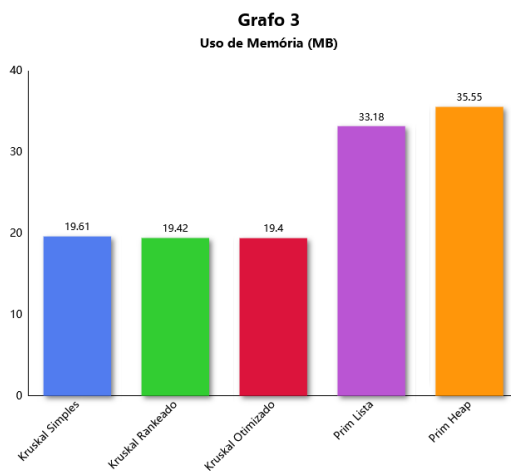


Figura 9. Memória - Grafo 3

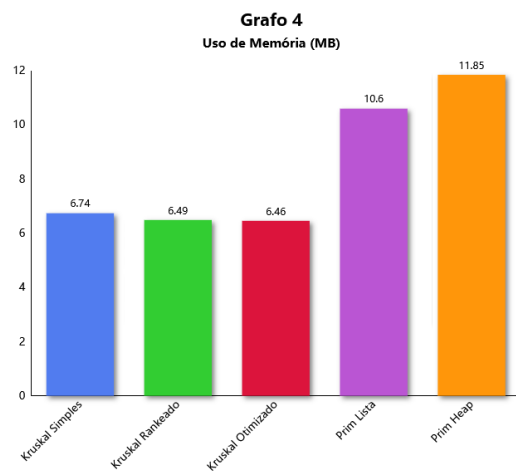


Figura 10. Memória - Grafo 4

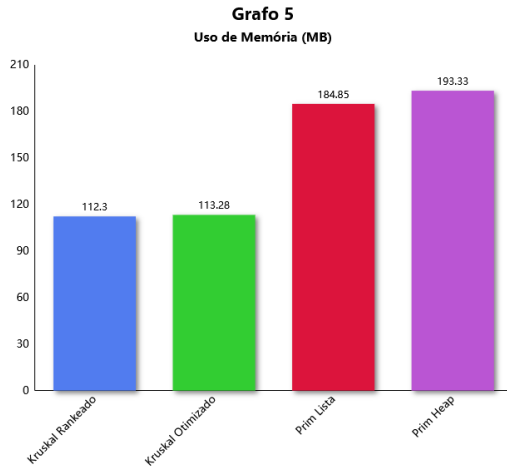


Figura 11. Memória - Grafo 5

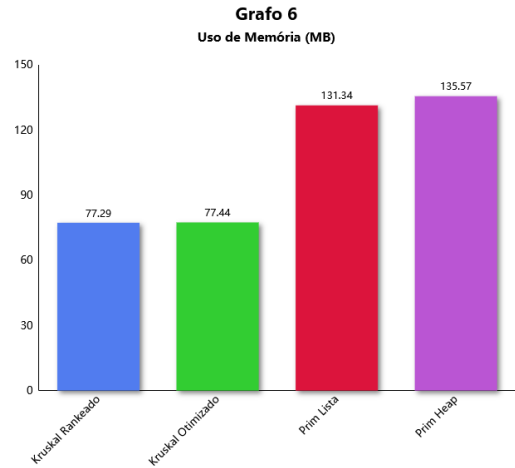


Figura 12. Memória - Grafo 6

Nota: O Prim Heap apresenta consistentemente o maior consumo de memória em todas as instâncias.

4 Discussão dos Resultados

A seguir, discutimos os resultados obtidos à luz da complexidade teórica dos algoritmos e das características das implementações realizadas.

4.1 Comparativo: Variações do Kruskal

A principal observação referente ao algoritmo de Kruskal foi a inviabilidade da versão *Simples* (sem otimizações) para grafos profundos. Como observado nos resultados, esta versão falhou nos grafos 5 e 6. Isso ocorre porque a implementação ingênua do *Union-Find* permite a criação de árvores desbalanceadas, o que, combinado com uma função *find* recursiva sem compressão de caminho, leva rapidamente ao estouro da pilha de recursão (*RecursionError*) do Python.

Entre as versões funcionais, *Kruskal Rank* e *Kruskal Otimizado*, a diferença de desempenho foi desprezível. O uso apenas do *Union by Rank* já foi suficiente para garantir a eficiência logarítmica das operações, tornando o ganho adicional da *Path Compression* pouco perceptível nestas instâncias específicas.

4.2 Comparativo: Prim Lista vs. Prim Heap

Este foi o comparativo mais impactante do estudo. O algoritmo *Prim Lista* demonstrou claramente a limitação da complexidade $O(N^2)$. Enquanto em grafos pequenos (Grafo 1) a diferença foi de apenas alguns segundos, no Grafo 5 a busca linear tornou o algoritmo quase 400 vezes mais lento que a versão com Heap (1170s contra 2,87s).

O *Prim Heap*, com complexidade $O(M \log N)$, comportou-se de maneira escalável, provando ser a única implementação viável do algoritmo de Prim para as instâncias de grande porte fornecidas neste trabalho.

4.3 Prim vs. Kruskal

No confronto direto entre as melhores versões de cada algoritmo (Kruskal Otimizado vs. Prim Heap), o algoritmo de Kruskal levou vantagem nos dois quesitos avaliados:

- **Tempo:** O Kruskal foi ligeiramente mais rápido na maioria das instâncias (ex: 1,48s contra 1,99s no Grafo 6).
- **Memória:** O Kruskal foi significativamente mais econômico. No Grafo 5, consumiu cerca de 113 MB, contra 193 MB do Prim Heap.

O maior consumo de memória do Prim Heap justifica-se pela necessidade de manter a estrutura de fila de prioridade e os dicionários auxiliares de pesos e predecessores, enquanto o Kruskal opera de forma eficiente com vetores simples e a lista de arestas.

5 Conclusão

O objetivo deste trabalho de avaliar e comparar os algoritmos de Prim e Kruskal foi plenamente atingido. Os experimentos realizados nas seis instâncias de grafos permitem concluir que:

1. A escolha da estrutura de dados é mais crítica que a escolha do algoritmo em si. A substituição da busca linear pelo Heap no algoritmo de Prim foi o fator determinante para viabilizar sua execução em grandes instâncias.
2. Para o tipo de grafo testado (esparso e de grande porte), o algoritmo de **Kruskal com Union by Rank** apresentou o melhor custo-benefício, aliando o menor consumo de memória a um tempo de execução competitivo.
3. Implementações recursivas ingênuas em Python (Kruskal Simples) possuem limitações severas de estabilidade e não devem ser utilizadas em ambientes de produção para grafos desconhecidos.

Como trabalhos futuros, sugere-se a expansão dos experimentos para incluir outras linguagens de programação, visando comparar o impacto do ambiente de execução no desempenho final. Além disso, seria interessante analisar o comportamento dos algoritmos em grafos com diferentes características e níveis de densidade, para verificar se as conclusões obtidas se mantêm em cenários mais diversificados.